

# Algorithm Analysis & Complexity Theory

## Sample Exam: All Topics

Generated: November 03, 2025

Difficulty Levels: Foundation, Conceptual, Application, Advanced

### Question 1 [Difficulty: Foundation]

Prove that  $3n^2 + 5n + 2 = O(n^2)$ . Use the formal definition of Big-O notation.

#### Solution:

To prove  $f(n) = O(g(n))$ , we need to show:  $\exists c > 0, n_0 > 0$  such that  $f(n) \leq c \cdot g(n)$  for all  $n \geq n_0$

Let  $f(n) = 3n^2 + 5n + 2$  and  $g(n) = n^2$

For  $n \geq 1$ :

- $3n^2 \leq 3n^2$
- $5n \leq 5n^2$  (since  $n \leq n^2$  for  $n \geq 1$ )
- $2 \leq 2n^2$  (since  $1 \leq n^2$  for  $n \geq 1$ )

Therefore:  $3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$

Choose  $c = 10$  and  $n_0 = 1$

Then  $f(n) \leq 10 \cdot g(n)$  for all  $n \geq 1$

$\therefore 3n^2 + 5n + 2 = O(n^2)$  ■

## Question 2 [Difficulty: Conceptual]

Show that  $n \log n \neq O(n)$ . Prove by contradiction.

### Solution:

Proof by contradiction:

Assume  $n \log n = O(n)$

Then  $\exists c > 0, n_0 > 0$  such that  $n \log n \leq cn$  for all  $n \geq n_0$

Dividing both sides by  $n$  (valid for  $n > 0$ ):

$\log n \leq c$  for all  $n \geq n_0$

But  $\log n$  is unbounded as  $n \rightarrow \infty$

For any constant  $c$ , we can find  $n_1$  such that  $\log n_1 > c$

(e.g.,  $n_1 = 2^{(c+1)}$ )

This contradicts our assumption that  $\log n \leq c$  for all  $n \geq n_0$

Therefore, our assumption was false

$\therefore n \log n \neq O(n)$  ■

### Question 3 [Difficulty: Conceptual]

Solve the recurrence  $T(n) = 2T(n/2) + n$  using the recursion tree method.

#### Solution:

Recursion Tree Method:

Level 0:  $n$  Cost:  $n$

Level 1:  $2 \cdot (n/2)$  Cost:  $n$

Level 2:  $4 \cdot (n/4)$  Cost:  $n$

Level 3:  $8 \cdot (n/8)$  Cost:  $n$

...

Level  $i$ :  $2^i \cdot (n/2^i)$  Cost:  $n$

Tree height:  $\log_2(n)$  (when we reach  $T(1)$ )

Total cost = sum of costs at each level

$= n + n + n + \dots + n$  ( $\log n$  times)

$= n \cdot \log n$

Therefore,  $T(n) = \Theta(n \log n)$  ■

## Question 4 [Difficulty: Application]

Use the Master Theorem to solve  $T(n) = 4T(n/2) + n^2$ .

### Solution:

Master Theorem for  $T(n) = aT(n/b) + f(n)$ :

Given:  $T(n) = 4T(n/2) + n^2$

-  $a = 4$

-  $b = 2$

-  $f(n) = n^2$

Calculate:  $\log_b(a) = \log_2(4) = 2$

Compare  $f(n) = n^2$  with  $n^{\log_b(a)} = n^2$ :

Case 2 applies:  $f(n) = \Theta(n^{\log_b(a)} \cdot \log^k(n))$  where  $k = 0$

Therefore:  $T(n) = \Theta(n^2 \log n)$  ■

## Question 5 [Difficulty: Conceptual]

Explain the optimal substructure property of the longest common subsequence (LCS) problem.

### Solution:

Optimal Substructure of LCS:

Let  $X = x_1x_2 \dots x_n$  and  $Y = y_1y_2 \dots y_m$  be two sequences

Let  $L[i, j]$  = length of LCS of  $X[1..i]$  and  $Y[1..j]$

Property:

If  $x_i = y_j$ :

$$L[i, j] = L[i-1, j-1] + 1$$

If  $x_i \neq y_j$ :

$$L[i, j] = \max(L[i-1, j], L[i, j-1])$$

Proof that this is optimal:

Case 1: If  $x_i = y_j$  and both are in the LCS

- The LCS must end with  $x_i = y_j$
- Remove this character: remaining is LCS of  $X[1..i-1]$  and  $Y[1..j-1]$
- If it wasn't optimal, we could construct a longer LCS - contradiction

Case 2: If  $x_i \neq y_j$

- At least one is not in the LCS
- LCS is either LCS of  $X[1..i-1], Y[1..j]$  or  $X[1..i], Y[1..j-1]$
- We take the maximum of both

This recursive structure leads to the DP solution ■

## Question 6 [Difficulty: Advanced]

Prove the correctness of Dijkstra's algorithm for finding shortest paths.

### Solution:

Dijkstra's Algorithm Correctness Proof:

Claim: When a vertex  $v$  is added to the set  $S$  of processed vertices,  $d[v]$  equals the shortest path distance from source  $s$  to  $v$ .

Proof by induction on  $|S|$ :

Base case:  $|S| = 1$

- $S = \{s\}$ ,  $d[s] = 0$
- Shortest path from  $s$  to  $s$  is  $0$  ✓

Inductive hypothesis: Assume true for  $|S| = k$

Inductive step: Prove for  $|S| = k+1$

- Let  $u$  be the next vertex added to  $S$
- $u$  has minimum  $d[u]$  among all vertices not in  $S$

We need to show:  $d[u]$  is the true shortest path distance

Proof by contradiction:

- Suppose  $\exists$  shorter path  $P$  from  $s$  to  $u$
- Let  $y$  be the first vertex on  $P$  not in  $S$
- Let  $x$  be the predecessor of  $y$  on  $P$  ( $x$  is in  $S$ )

Since edge weights are non-negative:

- $\text{distance}(s \rightarrow y \rightarrow u) \geq \text{distance}(s \rightarrow y)$
- By IH,  $d[x] = \text{distance}(s \rightarrow x)$
- $d[y] \leq d[x] + \text{weight}(x, y) = \text{distance}(s \rightarrow y)$

But  $u$  was chosen because  $d[u] \leq d[y]$

Therefore:  $d[u] \leq d[y] \leq \text{distance}(s \rightarrow y) \leq \text{distance}(s \rightarrow u \text{ via } P)$

This contradicts our assumption that  $P$  is shorter

$\therefore d[u]$  is the true shortest path distance ■



## Question 7 [Difficulty: Conceptual]

Prove that if  $P = NP$ , then every NP-complete problem can be solved in polynomial time.

### Solution:

Proof:

Definitions:

- $P$  = class of problems solvable in polynomial time
- $NP$  = class of problems verifiable in polynomial time
- NP-complete = hardest problems in  $NP$  (all  $NP$  problems reduce to them)

Assume:  $P = NP$

Let  $L$  be any NP-complete problem

By definition of NP-complete:

- (1)  $L \in NP$
- (2) For every problem  $L' \in NP$ , there exists a polynomial-time reduction from  $L'$  to  $L$  ( $L' \leq_p L$ )

Since  $P = NP$ :

- $L \in NP$  implies  $L \in P$
- Therefore,  $L$  can be solved in polynomial time

For any other NP-complete problem  $L'$ :

- $L' \leq_p L$  (by definition of NP-complete)
- $L$  can be solved in polynomial time
- A polynomial-time reduction composed with a polynomial-time algorithm gives a polynomial-time algorithm
- Therefore,  $L' \in P$

Conclusion: If  $P = NP$ , then every NP-complete problem is in  $P$  ■

Note: This is why  $P$  vs  $NP$  is so important - solving one NP-complete problem in polynomial time would solve them all!